



CYBERSTER PURPLE TEAM INTERNSHIP

Batch 1 — Month 1: Network Security & Protocol Analysis



WEEK 1 TECHNICAL REPORT

Network Security Lab Foundations & Protocol Deep-Dive

Intern Name	Shakera Karolia
Roll Number	CSI-B1-516
Program	Cyberster Internship Program — Batch 1
Team	Purple Team
Instructor	Muhammad Saad
Report Date	08 March 2026
Week	Week 1 of 12
Module	Month 1: Network Security & Protocol Analysis

CONFIDENTIAL — For Cyberster Internal Review Only

Executive Summary

This report documents the technical work completed during Week 1 of the Cyberster Purple Team Internship Program (Batch 1). The week focused on establishing foundational network security competencies through two practical tasks: constructing an isolated virtual laboratory environment and performing live network packet analysis using industry-standard tools.

Both tasks were completed successfully. A fully functional hybrid lab was built using Oracle VirtualBox 7.2.6, comprising a Kali Linux 2025.4 attacker machine and an Ubuntu 24.04 victim machine, networked via an isolated Internal Network adapter. Inter-machine connectivity was verified through successful ICMP ping tests with zero packet loss. Network traffic was subsequently captured and analysed using Wireshark 4.6.0, with the TCP Three-Way Handshake and common protocol headers documented in detail.

Task	Status	Evidence
Task 01: Virtualization & Network Topology	Completed	VirtualBox screenshots, ping output, ip addr confirmation
Task 02: Packet Analysis & Protocol Deep-Dive	Completed	Wireshark captures: ICMP, TCP handshake, protocol header expansion

TASK 01 — VIRTUALIZATION & NETWORK TOPOLOGY SETUP

1.1 Objective

To construct a professional hybrid laboratory environment consisting of isolated attacker and victim virtual machines, configured on a strictly internal network to prevent accidental exposure to external systems. This mirrors enterprise-grade security simulation environments used by SOC and penetration testing teams.

1.2 Technical Background

Virtualization & Hypervisors

Virtualization is the process of creating software-based representations of physical computing hardware. A hypervisor acts as the management layer between the host physical machine and guest virtual machines (VMs), enabling multiple isolated operating systems to run concurrently on a single physical host.

Oracle VirtualBox functions as a Type 2 hypervisor, running on top of an existing host operating system. This approach is well-suited for lab environments due to its flexibility, cost (free and open-source), and extensive networking configuration options.

Internal Network Architecture

VirtualBox supports multiple network modes. For this lab, the Internal Network mode was selected. This mode creates a completely isolated virtual switch that allows VMs to communicate with each other while preventing any traffic from reaching the host machine or the external internet. This is critical for safe execution of offensive security simulations.

1.3 Tools & Environment

Component	Specification
Hypervisor	Oracle VirtualBox 7.2.6 (amd64)
Attacker VM	Kali Linux 2025.4 (Pre-built VirtualBox image)
Victim VM	Ubuntu 24.04 LTS (64-bit)
Network Mode	Internal Network — Name: intnet
Attacker IP	192.168.10.1/24
Victim IP	192.168.10.2/24
Host OS	Windows 11

1.4 Implementation Steps

Step 1: VirtualBox Installation

Oracle VirtualBox 7.2.6 was downloaded from the official Oracle website and installed on the Windows 11 host machine. During installation, all networking components were installed, including VirtualBox Bridged Networking

and VirtualBox Host-Only Networking adapters. A temporary network disconnection warning was acknowledged, which is standard behaviour during installation of the virtual network drivers.

Step 2: Kali Linux VM Deployment

The Kali Linux 2025.4 pre-built VirtualBox image was downloaded from the official Kali Linux website (kali.org). The image was delivered as a compressed 7-Zip archive (3.54 GB compressed, expanding to approximately 15.8 GB). After extraction using WinRAR, the VM was added to VirtualBox via Machine → Add, selecting the .vbox configuration file from the extracted folder.

Step 3: Ubuntu VM Installation

An Ubuntu 24.04 LTS virtual machine was created manually within VirtualBox, configured with 4,485 MB RAM, 25 GB virtual disk, and VBoxSVGA graphics controller. Ubuntu was installed from the ISO image, providing the victim machine for the lab environment.

Step 4: Network Configuration

Both virtual machines were configured to use the Internal Network adapter, with the network name set to intnet on both VMs. This ensures the machines can communicate with each other while remaining completely isolated from the host machine and external internet.

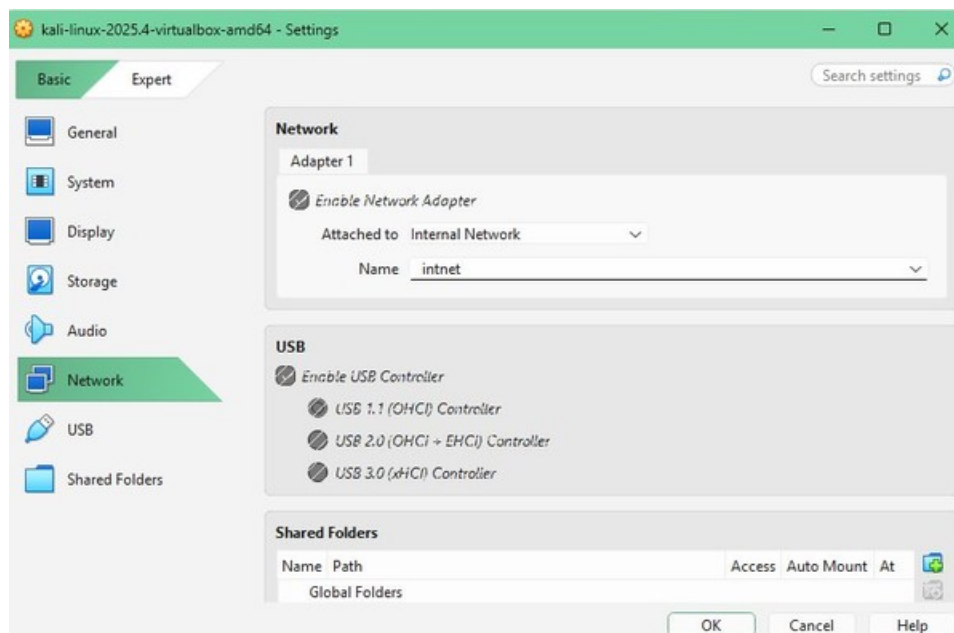


Figure 1: Kali Linux VM — Internal Network Configuration (VirtualBox Settings)

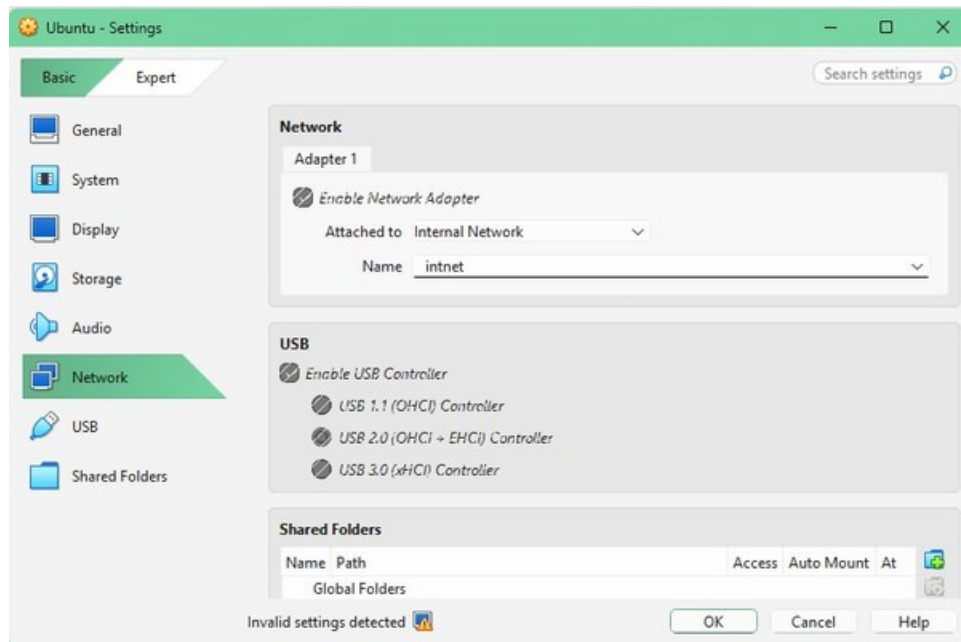


Figure 2: Ubuntu VM — Internal Network Configuration (VirtualBox Settings)

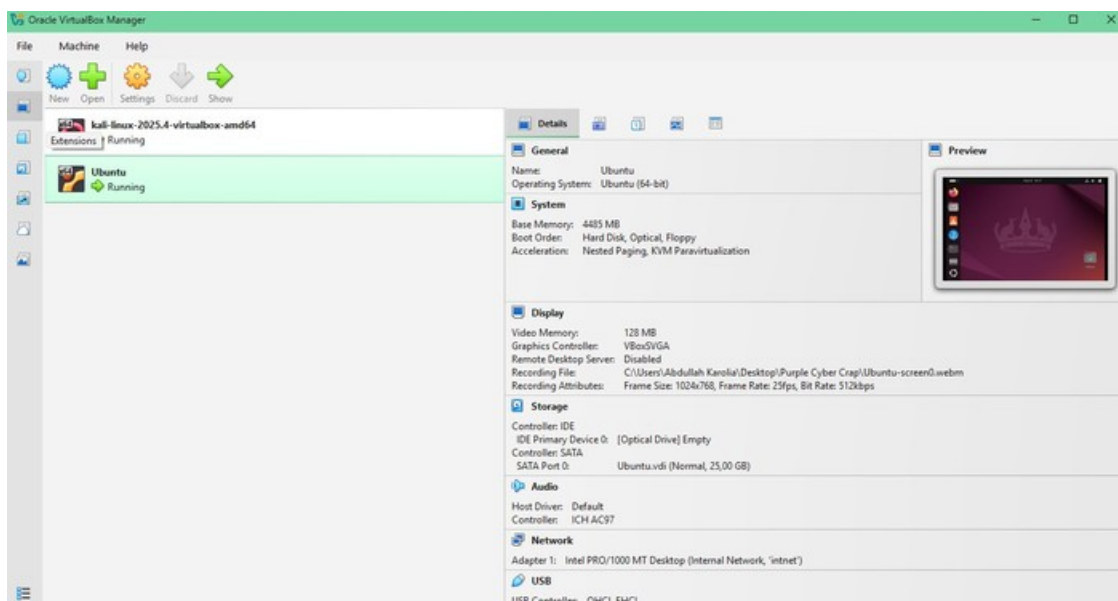


Figure 3: VirtualBox Manager showing both VMs running simultaneously

Step 5: Static IP Assignment

Static IP addresses were assigned to both virtual machines to ensure stable, predictable addressing for the lab environment. NetworkManager was used on Kali Linux to configure the persistent IP address, ensuring the configuration survived reboots.

```
# Kali Linux (Attacker) — 192.168.10.1
sudo nmcli con mod "Wired connection 1" ipv4.addresses 192.168.10.1/24 ipv4.method manual
sudo nmcli con up "Wired connection 1"
# Ubuntu (Victim) — 192.168.10.2
sudo ip addr add 192.168.10.2/24 dev enp0s3
```

```

kali@kali: ~
Session Actions Edit View Help
64 bytes from 192.168.10.2: icmp_seq=6 ttl=64 time=1.19 ms
64 bytes from 192.168.10.2: icmp_seq=7 ttl=64 time=1.09 ms
64 bytes from 192.168.10.2: icmp_seq=8 ttl=64 time=1.32 ms
64 bytes from 192.168.10.2: icmp_seq=9 ttl=64 time=1.19 ms
64 bytes from 192.168.10.2: icmp_seq=10 ttl=64 time=1.33 ms
64 bytes from 192.168.10.2: icmp_seq=11 ttl=64 time=1.01 ms
64 bytes from 192.168.10.2: icmp_seq=12 ttl=64 time=1.18 ms
64 bytes from 192.168.10.2: icmp_seq=13 ttl=64 time=1.11 ms
64 bytes from 192.168.10.2: icmp_seq=14 ttl=64 time=0.857 ms
64 bytes from 192.168.10.2: icmp_seq=15 ttl=64 time=0.955 ms
64 bytes from 192.168.10.2: icmp_seq=16 ttl=64 time=1.46 ms
^C
— 192.168.10.2 ping statistics —
16 packets transmitted, 16 received, 0% packet loss, time 15014ms
rtt min/avg/max/mdev = 0.857/1.282/2.233/0.341 ms

kali@kali:~$ sudo nmcli con up "Wired connection 1" ipv4.address 192.168.18.1/24 ipv4
[sudo] password for kali:
Error: invalid extra argument 'ipv4.address'.

kali@kali:~$ sudo nmcli con up "Wired connection 1" ipv4.addresses 192.168.18.1/24 ipv4.method manual
Error: invalid extra argument 'ipv4.addresses'.

kali@kali:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:03:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.1/24 brd 192.168.255.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::e0c5:41bb:b0f0:d6b7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

kali@kali:~$

```

Figure 4: Kali Linux — `ip addr show eth0` confirming 192.168.10.1/24 assignment

1.5 Connectivity Verification

Upon completion of the network configuration, inter-VM connectivity was verified using the ICMP ping utility. A sustained ping test was conducted from Kali Linux (attacker) to Ubuntu (victim), yielding 16 packets transmitted with 16 received and 0% packet loss, confirming stable communication between the machines over the Internal Network.

```

kali@kali: ~
Session Actions Edit View Help
64 bytes from 192.168.10.2: icmp_seq=6 ttl=64 time=1.19 ms
64 bytes from 192.168.10.2: icmp_seq=7 ttl=64 time=1.09 ms
64 bytes from 192.168.10.2: icmp_seq=8 ttl=64 time=1.32 ms
64 bytes from 192.168.10.2: icmp_seq=9 ttl=64 time=1.19 ms
64 bytes from 192.168.10.2: icmp_seq=10 ttl=64 time=1.33 ms
64 bytes from 192.168.10.2: icmp_seq=11 ttl=64 time=1.01 ms
64 bytes from 192.168.10.2: icmp_seq=12 ttl=64 time=1.18 ms
64 bytes from 192.168.10.2: icmp_seq=13 ttl=64 time=1.11 ms
64 bytes from 192.168.10.2: icmp_seq=14 ttl=64 time=0.857 ms
64 bytes from 192.168.10.2: icmp_seq=15 ttl=64 time=0.955 ms
64 bytes from 192.168.10.2: icmp_seq=16 ttl=64 time=1.46 ms
^C
— 192.168.10.2 ping statistics —
16 packets transmitted, 16 received, 0% packet loss, time 15014ms
rtt min/avg/max/mdev = 0.857/1.282/2.233/0.341 ms

kali@kali:~$ sudo nmcli con up "Wired connection 1" ipv4.address 192.168.18.1/24 ipv4
[sudo] password for kali:
Error: invalid extra argument 'ipv4.address'.

kali@kali:~$ sudo nmcli con up "Wired connection 1" ipv4.addresses 192.168.18.1/24 ipv4.method manual
Error: invalid extra argument 'ipv4.addresses'.

kali@kali:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:03:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.1/24 brd 192.168.255.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::e0c5:41bb:b0f0:d6b7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

kali@kali:~$

```

Figure 5: Successful ping from Kali (192.168.10.1) to Ubuntu (192.168.10.2) — 0% packet loss

1.6 Learning Outcomes

- Gained practical experience deploying and managing virtual machines using Oracle VirtualBox as a Type 2 hypervisor.
- Understood the distinction between network modes in VirtualBox, specifically the security implications of the Internal Network mode for isolated lab environments.
- Learned to configure static IP addresses persistently using NetworkManager (nmcli) on Kali Linux.

- Successfully constructed a controlled sandbox capable of executing high-risk security simulations without risk of external exposure.

TASK 02 — PACKET ANALYSIS & PROTOCOL DEEP-DIVE

2.1 Objective

To utilise Wireshark, an industry-standard network protocol analyser, to capture and inspect live network traffic between the attacker and victim virtual machines. The task required documenting the TCP Three-Way Handshake mechanism and identifying the forensic significance of common protocol headers at multiple OSI layers.

2.2 Technical Background

The OSI Model & Protocol Layering

Network communication is structured according to the OSI (Open Systems Interconnection) model, which defines seven layers of abstraction. Each layer adds its own header information to data as it is transmitted, a process known as encapsulation. Packet analysis tools like Wireshark allow analysts to inspect these headers at each layer, which is fundamental to network forensics and intrusion detection.

TCP Three-Way Handshake

The Transmission Control Protocol (TCP) is a connection-oriented, stateful protocol that guarantees reliable data delivery. Before any application data is exchanged, TCP requires the establishment of a connection through a three-step process known as the Three-Way Handshake:

- SYN (Synchronise) — The client sends a TCP segment with the SYN flag set, indicating its intent to establish a connection and sharing its initial sequence number.
- SYN-ACK (Synchronise-Acknowledge) — The server responds with both the SYN and ACK flags set, acknowledging the client's sequence number and sharing its own.
- ACK (Acknowledge) — The client sends a final ACK segment, confirming receipt of the server's sequence number. The connection is now established.

Understanding this handshake is critical in cybersecurity as it forms the basis for detecting SYN flood attacks (a form of denial-of-service), port scanning techniques such as TCP Connect and SYN scans, and session hijacking attempts.

2.3 Tools Used

Tool	Purpose
Wireshark 4.6.0	Live packet capture and protocol analysis
ping (ICMP)	Traffic generation for ICMP capture demonstration
nmap -sT	TCP Connect scan to generate TCP Three-Way Handshake traffic
netcat (nc)	TCP listener and connection tool for handshake generation

2.4 Capture Methodology

Initiating the Capture

Wireshark was launched on the Kali Linux attacker machine using elevated privileges (sudo wireshark). The eth0 interface was selected for capture, as this interface is bound to the Internal Network adapter where all inter-VM communication occurs. Packet capture was initiated, and live traffic immediately began populating the capture window.

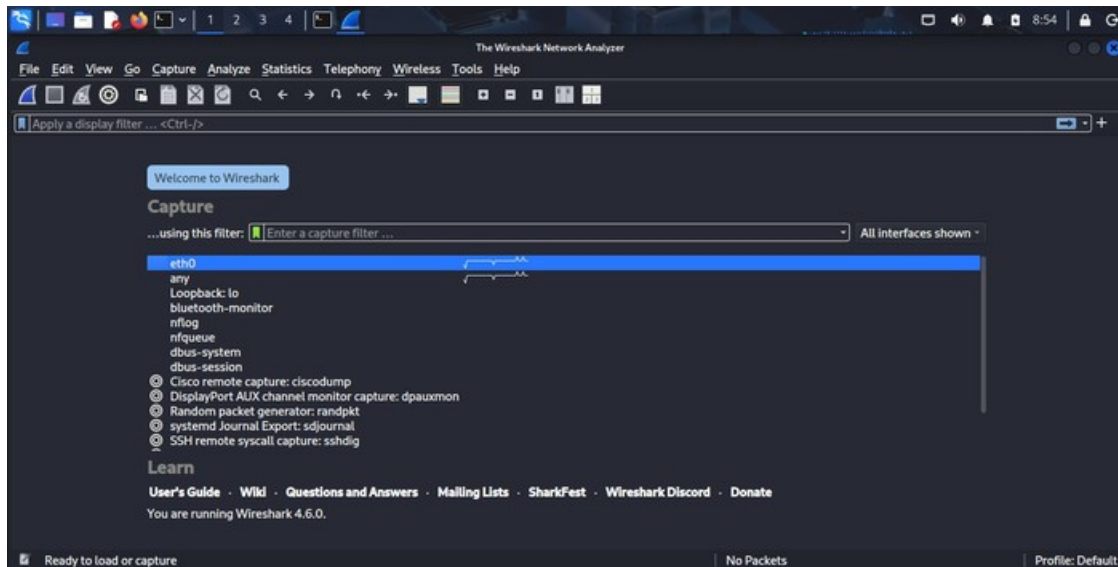


Figure 6: Wireshark 4.6.0 — Interface selection showing eth0 with live traffic activity

ICMP Traffic Analysis

With ping running between the two VMs, an icmp display filter was applied in Wireshark to isolate only ICMP packets. The capture clearly showed the alternating request and reply pattern characteristic of ping traffic, with all packets travelling between 192.168.10.1 (Kali) and 192.168.10.2 (Ubuntu).

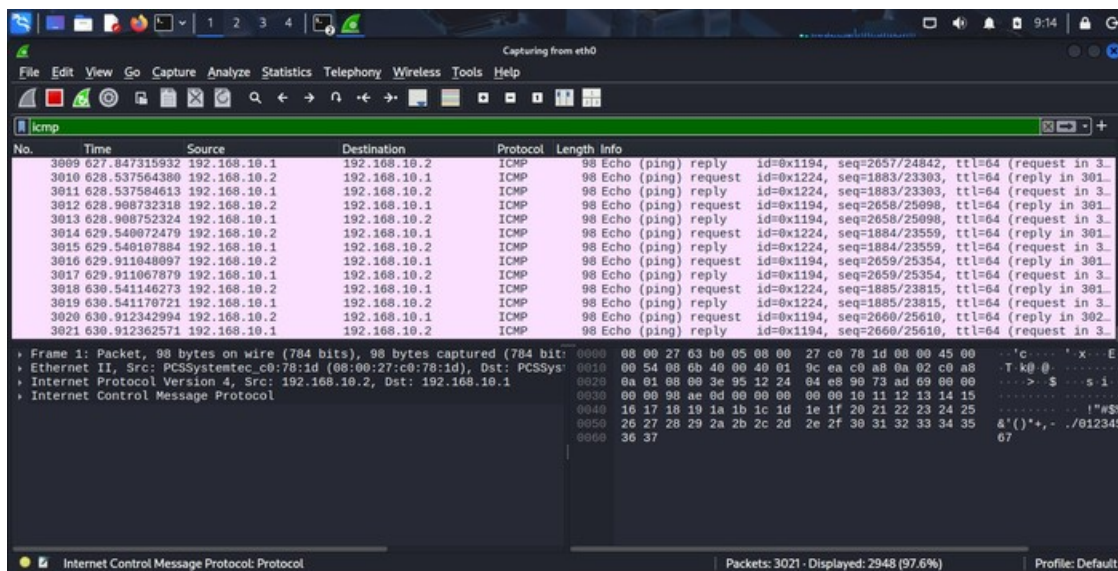


Figure 7: Wireshark ICMP filter — Echo request/reply packets between lab machines

2.5 Protocol Header Analysis

Ethernet II (Layer 2)

The Data Link layer header contains the source and destination MAC (Media Access Control) addresses of the communicating network interfaces. In the capture, the following MAC addresses were identified:

- Source MAC: 08:00:27:63:b0:05 (Kali Linux VM, Ubuntu victim sender)
- Destination MAC: 08:00:27:c0:78:1d (Ubuntu VM)
- EtherType: IPv4 (0x0800) — indicating the payload is an IP packet

Internet Protocol Version 4 (Layer 3)

The Network layer header contains addressing and routing information. Key fields observed in the capture:

- Version: 4 (IPv4)
- Header Length: 20 bytes (standard, no IP options)
- Total Length: 84 bytes
- Flags: 0x02 — Don't Fragment bit set
- Time to Live (TTL): 64 — standard Linux default, decremented at each router hop
- Protocol: ICMP (1) — identifying the encapsulated protocol
- Source Address: 192.168.10.2 | Destination Address: 192.168.10.1

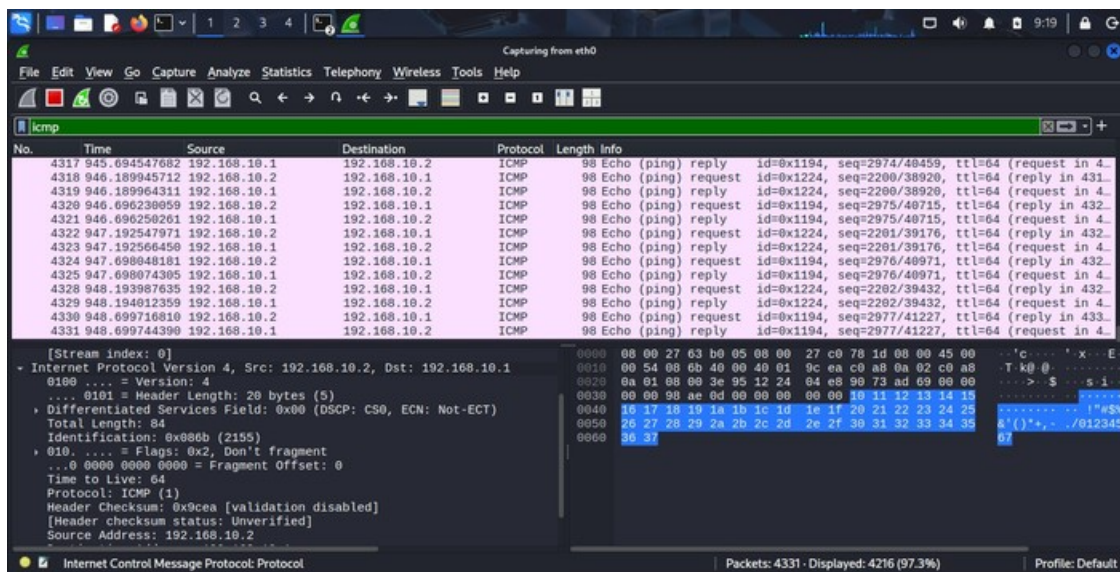


Figure 8: Wireshark — Fully expanded IP header showing all Layer 3 fields

2.6 TCP Three-Way Handshake Capture

To capture the TCP Three-Way Handshake, a TCP connection was established between Kali and Ubuntu using netcat on port 4444, followed by an nmap TCP Connect scan. A tcp display filter was applied in Wireshark, which revealed the complete handshake sequence. The three packets are clearly identified by their TCP flags in the Info column.

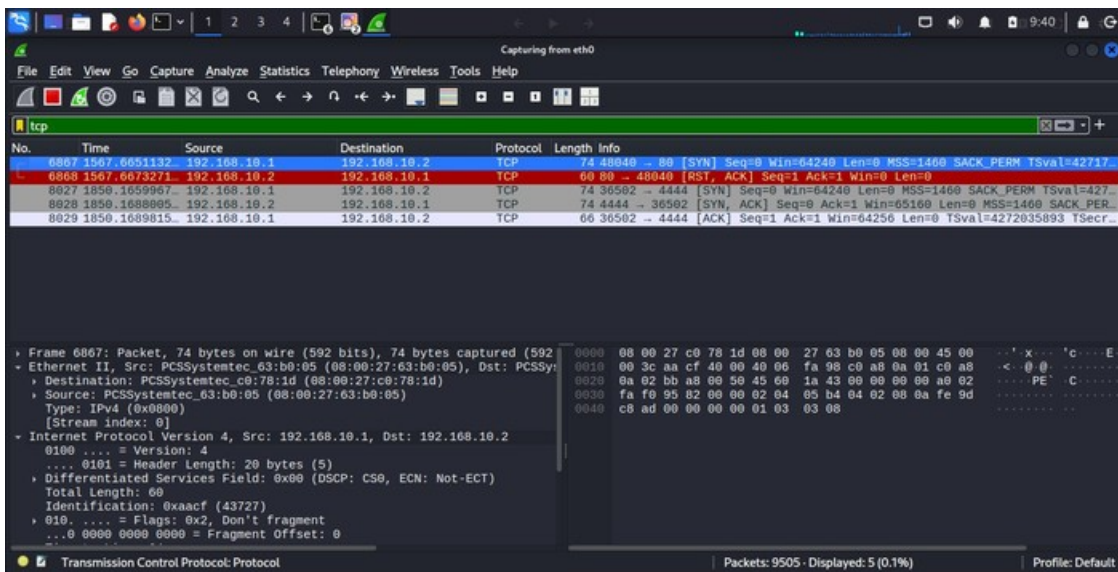


Figure 9: Wireshark TCP filter — SYN, SYN-ACK, and ACK packets confirming Three-Way Handshake (packets 8027–8029)

Handshake Packet Analysis

Step	Source	Destination	TCP Flag	Meaning
1	192.168.10.1 (Kali)	192.168.10.2 (Ubuntu)	[SYN]	"I want to connect"
2	192.168.10.2 (Ubuntu)	192.168.10.1 (Kali)	[SYN, ACK]	"Acknowledged, ready"
3	192.168.10.1 (Kali)	192.168.10.2 (Ubuntu)	[ACK]	"Connection confirmed"

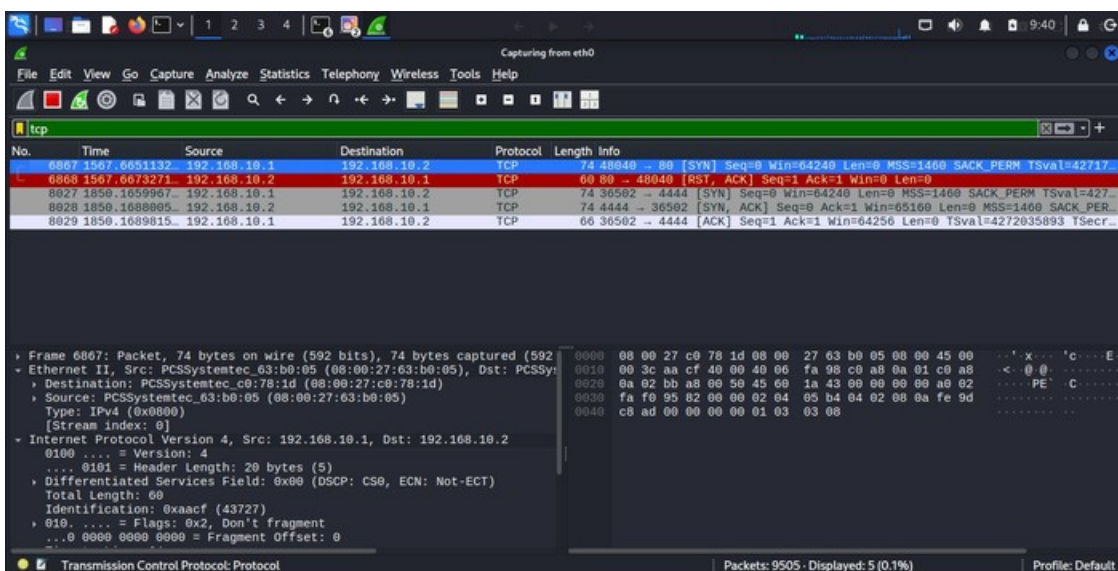


Figure 10: Packet 8027 expanded — TCP header showing Src Port: 36502, Dst Port: 4444, Seq=0 [SYN]

2.7 Learning Outcomes

- Developed proficiency in using Wireshark for live packet capture and protocol analysis.
- Understood the encapsulation of data across OSI layers by inspecting Ethernet II, IPv4, ICMP, and TCP headers in real captured traffic.

- Successfully identified and documented the TCP Three-Way Handshake from live traffic, understanding the forensic significance of each step.
- Recognised how protocol header fields such as TTL, flags, and sequence numbers can reveal information critical to network forensics and anomaly detection.
- Gained a practical understanding of how ICMP is exploited in network reconnaissance and why monitoring ICMP traffic is important for security operations.

Conclusion

Week 1 of the Cyberster Purple Team Internship successfully established both the practical infrastructure and conceptual foundations necessary for the programme's progression. The completion of both tasks demonstrates the ability to deploy and configure isolated virtual security lab environments and to perform meaningful network traffic analysis using professional tooling.

The internal network lab provides a stable, controlled environment for executing the more advanced offensive and defensive exercises scheduled in the coming weeks. The Wireshark analysis has reinforced a practical understanding of how data traverses the network stack and how protocol behaviour can be monitored and interpreted from a security perspective.

Both tasks directly support the programme's broader objective of building Purple Team proficiency: understanding how attackers operate at the network layer is fundamental to both offensive simulation and defensive detection engineering.

Submitted by: **Shakera Karolia** | Roll No: **CSI-B1-516**

Cyberster Purple Team Internship — Batch 1 | Week 1 Report | 08 March 2026